

Demarche complete de creation du **Departement 1** dans Supabase

Document d'approche technique pour la migration du Domaine Recrutement et Integration vers la base de donnees Supabase. Couvre l'architecture des 17 tables SQL, le mapping des 18 feuilles Excel, les politiques RLS multi-tenant, le schema detaille des 3 services, les interconnexions inter-domaines et le plan de migration en 5 phases.

PROJET

ADMINA-RH — Schema admina_rh

TECHNOLOGIE

Supabase • PostgreSQL 17.6 • Multi-tenant RLS

CONTENU

17 tables • 39 ENUMs • 68 politiques RLS

Table des matieres

1. Vue d'ensemble du Departement 1	3
1.1 Contexte et positionnement dans ADMINA-RH	3
1.2 Organisation structurelle : 3 services	3
1.3 Inventaire complet des 18 feuilles Excel	3
1.4 Architecture du Departement 1 (4 sections)	4
2. Architecture du schema Supabase pour le Departement 1	4
2.1 Hierarchie des schemas PostgreSQL	4
2.2 Colonnes systeme obligatoires (multi-tenant)	5
2.3 Mapping des types Excel vers PostgreSQL	5
2.4 Gestion des 39 listes de reference (ENUM)	6
3. Schema SQL detaille par service	6
3.1 Service 1.1 : Gestion Administrative du Personnel	6
3.1.1 Table <i>d1_demandes_recrutement</i>	6
3.1.2 Table <i>d1_base_candidats</i>	7
3.1.3 Tables connexes du Service 1.1	7
3.2 Service 1.2 : Organisation du Travail	8
3.3 Service 1.3 : Cartographie des Metiers	8
4. Politiques RLS et securite multi-tenant	8
4.1 Principe du Row Level Security	8
4.2 Les 5 roles RBAC + ABAC	8
4.3 Exemple de politique RLS	9
5. Relations et cles etrangeres	9
5.1 Relations intra-Domaine 1	9
5.2 Interconnexions avec les autres domaines	10
6. Plan de migration et etapes detaillees	11
6.1 Phase 0 : Preparation de l'environnement	11
6.2 Phase 1 : Types ENUM et tables de reference	11

6.3 Phase 2 : Creation des tables principales (17 tables)	12
6.4 Phase 3 : Index, triggers et fonctions	12
6.5 Phase 4 : Politiques RLS (17 tables)	12
6.6 Phase 5 : Donnees de test et validation	13
7. Estimation de la charge et roadmap	13
7.1 Volumetrie du Departement 1	13
7.2 Roadmap de mise en oeuvre	14
7.3 Risques et mitigations	14

1. Vue d'ensemble du Departement 1

1.1 Contexte et positionnement dans ADMINA-RH

Le Departement 1 constitue le pilier fondamental du systeme ADMINA-RH. Il est entierement dedie au **Recrutement et a l'Integration** des collaborateurs, deux processus qui conditionnent la qualite de l'ensemble de la gestion des ressources humaines. Dans une organisation multi-tenant hebergee sur Supabase (PostgreSQL 17.6, region eu-west-1, plan Pro), ce departement sera le premier a etre implemente dans le schema **admina_rh**, schema enfant de **admina**. Ce choix strategique s'explique par le fait que le recrutement est le point d'entree principal de tout collaborateur dans le systeme : chaque nouvelle embauche genere des donnees qui alimentent ensuite les 30 autres domaines (paie, temps de travail, formation, performance, etc.).

Le Departement 1 se decompose en **3 services** structurels, eux-memes appuyes sur **18 feuilles Excel** (dont 1 feuille de listes de reference et 1 tableau de bord). La Section 0 (Cadre strategique) et la Section 1 (Architecture generale) definissent le contexte organisationnel global, tandis que les Sections 2 a 4 detailent chaque service. L'ensemble couvre le cycle de vie complet d'un candidat, depuis la demande de recrutement initiale jusqu'au suivi post-embauche et a la gestion des stagiaires et saisonniers.

1.2 Organisation structurelle : 3 services

Le Departement 1 s'articule autour de trois services complementaires qui couvrent l'ensemble du processus de recrutement et d'integration. Chaque service possede ses propres tables SQL, mais partage des references communes (departements, postes, employes) via le schema admina_rh. Voici la decomposition detaillee :

Service	Intitule	Feuilles Excel	Tables SQL cibles
1.1	Gestion Administrative du Personnel	Feuilles 1 a 10 + TdB	7 tables principales
1.2	Organisation du Travail	Feuilles 11 a 14	4 tables principales
1.3	Cartographie des Metiers	Feuilles 15 a 18	4 tables principales

Tableau 1 : Decomposition des services du Departement 1

1.3 Inventaire complet des 18 feuilles Excel

Le fichier **Domaine1_Recrutement_Candidats.xlsx** contient 18 feuilles qui representent autant de tables SQL a creer dans le schema admina_rh. La feuille **_Lists** contient 39 listes de reference (enums PostgreSQL) qui seront transformees en types ENUM ou en tables de reference avec contraintes CHECK. Le tableau de bord (feuille 18) ne genere pas de table SQL mais sera recree sous forme de vues materialisees ou de requetes dashboard dans l'application frontend.

Feuille Excel	Table SQL	Colonnes	Lignes	Role
1-Demandes Recrutement	demandes_recrutement	20 colonnes	~95 lignes	Table principale
2-Base Candidats	base_candidats	37 colonnes	~93 lignes	Table principale

Feuille Excel	Table SQL	Colonnes	Lignes	Role
3-Planning Entretiens	planning_entretiens	16 colonnes	~95 lignes	Table principale
4-Grille Evaluation	grille_evaluation	20 colonnes	~97 lignes	Table principale
5-Verification References	verification_references	16 colonnes	~94 lignes	Table principale
6-Suivi Contrats	suivi_contrats	15 colonnes	~97 lignes	Table principale
7-Gestion Cabinets	gestion_cabinets	17 colonnes	~97 lignes	Table principale
8-Prevision Postes Offres	prevision_postes_offres	17 colonnes	~96 lignes	Table principale
9-Sources Recrutement	sources_recrutement	12 colonnes	~97 lignes	Table analytique
10-Analyse Couts	analyse_couts	13 colonnes	~97 lignes	Table analytique
11-Integration Employe	integration_employe	15 colonnes	~93 lignes	Table principale
12-Checklist Integration	checklist_integration	13 colonnes	~97 lignes	Table de suivi
13-Periode Essai	periode_essai	16 colonnes	~94 lignes	Table principale
14-Plan Accueil Formation	plan_accueil_formation	14 colonnes	~97 lignes	Table de suivi
15-Stagiaires	stagiaires	15 colonnes	~95 lignes	Table principale
16-Saisonniers Temporaires	saisonniers_temporaires	15 colonnes	~95 lignes	Table principale
17-Suivi Post-Embauche	suivi_post_embauche	13 colonnes	~93 lignes	Table de suivi
18-Tableau de Bord	(vue dashboard)	11 colonnes	~73 lignes	Vues / Requetes

Tableau 2 : Inventaire complet des 18 feuilles du Domaine 1

1.4 Architecture du Departement 1 (4 sections)

L'architecture du Departement 1, telle que documentee dans les fichiers Archi_Systeme_Dep1_*.pdf, se decompose en 4 sections hierarchiques. La Section 0 etablit le cadre strategique global du projet ADMINA-RH (organisation multi-tenant, stack technique, gouvernance). La Section 1 presente l'architecture generale du Departement 1 avec le modele de donnees partage, le flux d'authentification et l'architecture evenementielle. La Section 2 detaille le Service 1.1 (Gestion Administrative du Personnel) avec son modele ER, ses workflows et ses regles metier. La Section 3 decrit le Service 1.2 (Organisation du Travail) incluant le modele de donnees, les flux evenementiels et le cycle de vie du planning/pointage. Enfin, la Section 4 aborde le Service 1.3 (Cartographie des Metiers) avec le modele de donnees D21, les processus d'ecarts de competences et le cycle de vie des fiches de poste.

2. Architecture du schema Supabase pour le Departement 1

2.1 Hierarchie des schemas PostgreSQL

Dans Supabase, l'organisation des donnees suit le modele de schemas PostgreSQL. Le schema parent **admina** sert de conteneur pour tous les sous-projets de l'organisation. Le schema enfant **admina_rh** hebergera l'ensemble des 31 domaines RH (D1 a D31) du projet ADMINA-RH. Chaque table du Departement 1 sera prefixee par **d1_** pour

garantir l'unicite des noms a l'echelle du schema. Par exemple, la feuille 'Demandes Recrutement' devient la table `admina_rh.d1_demandes_recrutement`.

Cette convention de nommage offre plusieurs avantages : elle facilite la recherche de tables par domaine, evite les collisions de noms entre les 31 domaines, et permet d'implementer des politiques RLS (Row Level Security) specifiques par prefixe. De plus, les futures extensions (`admina_comptable`, `admina_commerce`) suivront la meme logique dans des schemas separes, assurant une isolation complete entre les sous-projets tout en preservant la possibilite de requetes inter-schemas lorsque c'est necessaire.

2.2 Colonnes systeme obligatoires (multi-tenant)

Chaque table du Departement 1 doit inclure un ensemble de colonnes systeme qui assurent le fonctionnement du modele multi-tenant, l'audit et le versionnage. Ces colonnes sont ajoutees automatiquement par le script de migration et ne sont pas presentes dans le fichier Excel source. Elles sont essentielles pour la securite des donnees (RLS), la tracabilite des operations et la gestion des conflits en cas de modifications concurrentes.

Colonne	Type PostgreSQL	Default	Role
<code>id</code>	UUID	<code>gen_random_uuid()</code>	Cle primaire unique
<code>tenant_id</code>	UUID	NOT NULL	Isolation multi-tenant (RLS)
<code>created_at</code>	TIMESTAMPTZ	<code>now()</code>	Horodatage de creation
<code>updated_at</code>	TIMESTAMPTZ	<code>now()</code>	Horodatage de derniere modification
<code>created_by</code>	UUID	<code>auth.uid()</code>	Auteur de la creation
<code>updated_by</code>	UUID	<code>auth.uid()</code>	Dernier editeur
<code>is_active</code>	BOOLEAN	<code>true</code>	Soft delete (archivage logique)
<code>version</code>	INTEGER	<code>1</code>	Optimistic locking

Tableau 3 : Colonnes systeme obligatoires pour chaque table

2.3 Mapping des types Excel vers PostgreSQL

La conversion des types de donnees Excel vers PostgreSQL requiert une attention particuliere pour garantir la coherence et la performance. Les dates Excel (stockees en format datetime avec heure 00:00:00) doivent etre converties en `DATE` pour les dates pures ou `TIMESTAMPTZ` pour les horodatages. Les montants en FCFA sont convertis en `BIGINT` pour eviter les problemes d'arrondi des types `FLOAT/REAL`. Les listes de reference (feuille `_Lists`) deviennent soit des types `ENUM` PostgreSQL, soit des tables de reference avec contraintes `CHECK`. Le tableau ci-dessous resume les regles de conversion appliquees a l'ensemble des 18 feuilles du Departement 1.

Type Excel	Exemple	Type PostgreSQL	Justification
Texte (identite)	Nom, Prenom, Email	<code>VARCHAR(n)</code>	Longueur calibree par colonne
Texte (notes)	Notes, Commentaires	<code>TEXT</code>	Longueur variable, pas de troncature

Type Excel	Exemple	Type PostgreSQL	Justification
Date	2025-01-05	DATE	Comparaisons et index plus rapides
Datetime	2025-01-25 10:00	TIMESTAMPTZ	Fuseau horaire Europe/Paris
Montant FCFA	350000	BIGINT	Precision entiere, pas d'arrondi
Entier	Annees Exp., Duree	INTEGER	Valeurs numeriques entieres
Decimal (score)	Score /20	NUMERIC(5,2)	Precision decimale
Pourcentage	Taux Transformation	NUMERIC(5,2)	Stocke en valeur brute (ex: 33.5)
Booleen	Oui/Non, Fait/Non fait	BOOLEAN	Conversion Oui->true, Non->>false
Liste (statut)	En cours, Cloturee	ENUM ou CHECK	Contrainte d'integrite
Telephone	699 55 44 33	VARCHAR(20)	Format libre, pas de validation

Tableau 4 : Regles de mapping des types Excel vers PostgreSQL

2.4 Gestion des 39 listes de reference (ENUM)

La feuille **_Lists** du fichier Excel contient 39 listes de reference qui alimentent les menus deroulants et les contraintes de validation de l'ensemble des 18 feuilles. Chaque liste sera transformee en un type ENUM PostgreSQL dans le schema `admina_rh`, avec le prefixe `d1_`. Par exemple, la liste 'statut_demande' devient le type `admina_rh.d1_statut_demande`. Les valeurs des listes sont nettoyees avant import : les espaces superflus sont supprimes, les accents sont conserves, et les caracteres speciaux sont echappes. Les types ENUM offrent l'avantage de garantir l'integrite referentielle au niveau de la base de donnees, sans necessiter de tables de reference supplementaires ni de jointures pour la validation.

Cependant, certains ENUMs devront etre partages entre plusieurs domaines. Par exemple, 'departement', 'type_contrat', 'civilete' et 'genre' sont utilises dans D1 mais aussi dans D2, D3, D4 et bien d'autres. Dans ce cas, ces types seront definis dans le schema `admina_rh` sans prefixe de domaine (par exemple `admina_rh.type_contrat`), et les tables du Departement 1 y feront reference directement. Cette strategie evite la duplication des types et facilite les interconnexions entre domaines.

3. Schema SQL detaille par service

3.1 Service 1.1 : Gestion Administrative du Personnel

Le Service 1.1 est le coeur operationnel du Departement 1. Il couvre l'ensemble du processus de recrutement depuis la demande initiale jusqu'a la signature du contrat, en passant par la gestion des candidats, les entretiens, l'evaluation, la verification des references et l'analyse des couts. Ce service comprend les 10 premieres feuilles Excel ainsi que le tableau de bord, soit 10 tables SQL principales et 2 tables analytiques.

3.1.1 Table `d1_demandes_recrutement`

Cette table est le point d'entree de tout processus de recrutement. Chaque demande est generee par un responsable de departement et suit un cycle de vie precis : creation, validation par la DRH, publication de l'offre, reception des

candidatures, selection, et cloture. La cle primaire est un UUID auto-genere, mais un identifiant metier `numero_demande` (format DR-YYYY-NNN) assure la traçabilité fonctionnelle. Le champ `tenant_id` est obligatoire pour la segregation multi-tenant, et la politique RLS garantit que chaque entreprise ne voit que ses propres demandes.

```
CREATE TABLE admina_rh.d1_demandes_recrutement (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES admina.tenants(id),
  numero_demande VARCHAR(20) UNIQUE NOT NULL,
  date_demande DATE NOT NULL DEFAULT CURRENT_DATE,
  departement VARCHAR(100) NOT NULL,
  poste_recherche VARCHAR(200) NOT NULL,
  type_poste admina_rh.d1_type_poste NOT NULL,
  type_contrat admina_rh.type_contrat NOT NULL,
  effectif_demande INTEGER NOT NULL DEFAULT 1,
  motif admina_rh.d1_motif_recrutement NOT NULL,
  date_besoin DATE,
  priorite admina_rh.d1_priorite NOT NULL DEFAULT 'Moyenne',
  statut admina_rh.d1_statut_demande NOT NULL DEFAULT 'En attente',
  date_pourvue DATE,
  responsable_demande VARCHAR(150) NOT NULL,
  role_responsable VARCHAR(100),
  cabinet_externe VARCHAR(200),
  budget_salaire BIGINT,
  cout_recrutement BIGINT,
  delai_jours INTEGER,
  notes TEXT,
  -- Colonnes systeme
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  created_by UUID DEFAULT auth.uid(),
  updated_by UUID DEFAULT auth.uid(),
  is_active BOOLEAN DEFAULT true,
  version INTEGER DEFAULT 1
);
```

3.1.2 Table d1_base_candidats

La base candidats est la table la plus riche du Departement 1 avec 37 colonnes. Elle centralise toutes les informations d'identité, de coordonnées, de formation et de compétences de chaque candidat. Le candidat peut être associé à plusieurs demandes de recrutement et à plusieurs entretiens via des relations foreign key. Les documents téléchargeables (CV, lettres, certificats) sont stockés dans Supabase Storage (bucket `admina-rh-documents`) et référencés par URL dans les champs correspondants. Les scores sont stockés en NUMERIC(5,2) pour permettre des évaluations à demi-points.

3.1.3 Tables connexes du Service 1.1

Outre les deux tables principales, le Service 1.1 comprend huit tables supplémentaires qui couvrent les aspects opérationnels et analytiques du recrutement. La table `d1_planning_entretiens` gère le calendrier des entretiens avec les évaluateurs, les lieux et les liens visioconférence. La table `d1_grille_evaluation` structure l'évaluation des candidats selon 5 critères pondérés sur 25 points. La table `d1_verification_references` assure le suivi des vérifications pré-embauche (diplôme, expérience, comportement, salaire déclaré). La table `d1_suivi_contrats` assure le suivi lifecycle des contrats de travail (CDI, CDD, intérim, stage). La table `d1_gestion_cabinets` gère les prestataires externes de recrutement avec leurs KPI (taux de transformation,

cout par recrutement). La table `d1_prevision_postes_offres` assure la planification previsionnelle des besoins en personnel. Enfin, les tables `d1_sources_recrutement` et `d1_analyse_couts` fournissent les indicateurs analytiques pour le pilotage de la fonction recrutement.

3.2 Service 1.2 : Organisation du Travail

Le Service 1.2 couvre le processus d'integration des nouveaux employes, depuis leur arrivee jusqu'a la fin de leur periode d'essai. Ce service comprend 4 tables SQL correspondant aux feuilles 11 a 14 du fichier Excel. La table `d1_integration_employe` suit l'etat d'avancement de l'integration (documents administratifs, formation securite, formation metier, equipement, visite des locaux). La table `d1_checklist_integration` decompose chaque integration en etapes granulaires avec des statuts individuels (Fait, En cours, Non applicable). La table `d1_periode_essai` assure le suivi des evaluations a mi-parcours et en fin de periode d'essai, avec les decisions associees (embauche confirmee, prolongation, rupture). La table `d1_plan_accueil_formation` planifie les modules de formation des nouveaux arrivees avec les formateurs, durees et evaluations.

3.3 Service 1.3 : Cartographie des Metiers

Le Service 1.3 etend le perimetre du Departement 1 aux populations non-salariees (stagiaires, saisonniers, temporaires) et au suivi post-embauche. Les 4 tables de ce service complètent le cycle de vie du collaborateur. La table `d1_stagiaires` gere les conventions de stage avec les etablissements, les tuteurs, les indemnites et les evaluations. La table `d1_saisonniers_temporaires` suit les contrats de travail saisonniers avec les couts associes. La table `d1_pipeline_candidatures` offre une vue Kanban du parcours candidat (CV reçu, pre-selection, entretien HR, test technique, offre, embauche). Enfin, la table `d1_suivi_post_embauche` assure le suivi a 1 mois, 3 mois et 6 mois avec les indicateurs de satisfaction, d'integration et de risque de depart.

4. Politiques RLS et securite multi-tenant

4.1 Principe du Row Level Security

Le Row Level Security (RLS) est le mecanisme fondamental de securite dans Supabase. Chaque table du Departement 1 sera protegee par des politiques RLS qui garantissent qu'un utilisateur ne peut acceder qu'aux lignes de son tenant (entreprise). Le principe est simple : chaque requete SQL est automatiquement filtre par une clause `WHERE tenant_id = current_setting('app.tenant_id')::uuid`. Le `tenant_id` est extrait du JWT de l'utilisateur authentifie et injecte dans un parametre de session PostgreSQL au moment de la connexion via un hook Supabase. Cela signifie que meme si un utilisateur tente de modifier manuellement une requete pour acceder aux donnees d'un autre tenant, la politique RLS bloquera l'accès au niveau de la base de donnees.

4.2 Les 5 roles RBAC + ABAC

Le systeme ADMINA-RH implemente un modele hybride RBAC (Role-Based Access Control) et ABAC (Attribute-Based Access Control) avec 5 roles principaux. Chaque role definit un ensemble de permissions specifiques sur les tables du Departement 1. Les roles sont verifiés dans le JWT de l'utilisateur et utilises par les politiques RLS pour determiner les actions autorisees (SELECT, INSERT, UPDATE, DELETE). Le role **admin_tenant** a un acces complet a toutes les donnees de son tenant. Le role **drh** peut lire et modifier toutes les tables RH. Le role **manager** peut creer des demandes de recrutement et consulter les candidats de son

departement. Le role **rh_agent** peut gerer les operations quotidiennes de recrutement mais pas modifier la structure des tables. Enfin, le role **employee** a un acces en lecture seule a ses propres donnees (candidature, contrat, evaluation).

Role	Demandes	Candidats	Entretiens	Contrats	Cabinets
admin_tenant	CRUD complet	CRUD complet	CRUD complet	CRUD complet	CRUD complet
drh	CRUD	CRUD	CRUD	CRUD	CRUD
manager	CR (son dept.)	R (son dept.)	R (son dept.)	R	R
rh_agent	CRUD	CRUD	CRUD	CRUD	R
employee	R (ses donnees)	R (ses donnees)	R (ses donnees)	R (son contrat)	Aucun

Tableau 5 : Matrice des permissions par role (CRUD)

4.3 Exemple de politique RLS

Voici un exemple concret de politique RLS pour la table `d1_demandes_recrutement`. La politique utilise le helper Supabase `auth.uid()` pour recuperer l'utilisateur courant et le parametre de session `app.tenant_id` pour filtrer par tenant. La politique de `SELECT` verifie a la fois l'appartenance au tenant et le role de l'utilisateur pour determiner s'il peut acceder a la ligne. Cette double verification (tenant + role) assure une securite maximale contre les fuites de donnees inter-tenants.

```
-- Activation RLS
ALTER TABLE admina_rh.d1_demandes_recrutement ENABLE ROW LEVEL SECURITY;

-- Politique SELECT : admin_tenant, drh, manager (son dept.), rh_agent, employee
CREATE POLICY sel_demandes ON admina_rh.d1_demandes_recrutement
FOR SELECT USING (
tenant_id = current_setting('app.tenant_id')::uuid
AND (
current_setting('app.role') IN ('admin_tenant','drh','rh_agent')
OR (current_setting('app.role') = 'manager'
AND departement = current_setting('app.departement'))
OR (current_setting('app.role') = 'employee'
AND responsable_demande = current_setting('app.user_name'))
)
);

-- Politique INSERT : admin_tenant, drh, rh_agent, manager
CREATE POLICY ins_demandes ON admina_rh.d1_demandes_recrutement
FOR INSERT WITH CHECK (
tenant_id = current_setting('app.tenant_id')::uuid
AND current_setting('app.role') IN ('admin_tenant','drh','rh_agent','manager')
);
```

5. Relations et cles etrangeres

5.1 Relations intra-Domaine 1

Les 17 tables du Departement 1 sont fortement interconnectes par des relations one-to-many et many-to-many. La demande de recrutement est l'entité centrale du Service 1.1 : elle est liee a plusieurs candidats (via la table de liaison candidats-demandes), a plusieurs entretiens, et a une ou plusieurs offres d'emploi. Le candidat est le noeud central de tout le processus : il est evalue (grille_evaluation), verifie (verification_references), puis eventuellement transforme en employe avec un contrat (suivi_contrats). L'employe est ensuite integre (integration_employe, checklist_integration), evalue en periode d'essai (periode_essai), forme (plan_accueil_formation) et suivi post-embauche (suivi_post_embauche). Le pipeline candidatures offre une vue synthetique du parcours candidat a travers tous ces stades.

Table source	Relation	Table cible	Cardinalite
d1_demandes_recrutement	genere	d1_planning_entretiens	1 : N
d1_base_candidats	passe	d1_planning_entretiens	1 : N
d1_base_candidats	est evalue dans	d1_grille_evaluation	1 : N
d1_base_candidats	est verifie dans	d1_verification_references	1 : N
d1_base_candidats	debouche sur	d1_suivi_contrats	1 : 1
d1_suivi_contrats	declenche	d1_integration_employe	1 : 1
d1_integration_employe	se decompose en	d1_checklist_integration	1 : N
d1_suivi_contrats	est suivi par	d1_periode_essai	1 : N
d1_integration_employe	suit	d1_plan_accueil_formation	1 : N
d1_base_candidats	transite dans	d1_pipeline_candidatures	1 : N
d1_suivi_contrats	est suivi par	d1_suivi_post_embauche	1 : 1
d1_demandes_recrutement	utilise	d1_gestion_cabinets	N : 1

Tableau 6 : Matrice des relations intra-Domaine 1

5.2 Interconnexions avec les autres domaines

Le Departement 1 est le domaine le plus interconnecte du projet ADMINA-RH. Il possede des liens directs avec au moins 15 autres domaines. La relation la plus forte est avec le Domaine 2 (Administration du Personnel) : chaque candidat embauche devient un employe dans le registre du personnel D2, et les informations d'identite et de coordonnees sont synchronisees via des triggers PostgreSQL. Le Domaine 3 (Paie et Remuneration) utilise les donnees de contrat (type, salaire, date debut) pour calculer les bulletins de paie. Le Domaine 4 (Temps de Travail) recupere les dates d'embauche pour initialiser les plannings de pointage. Le Domaine 5 (Conges et Absences) utilise les dates de contrat pour calculer les droits aux conges. Le Domaine 6 (Formation) recupere les donnees d'evaluation pour adapter les plans de formation. Le Domaine 7 (Performance) utilise les scores d'entretien comme baseline. Le Domaine 10 (Talents) recupere les profils candidats pour la gestion previsionnelle des emplois. Les domaines 11 (Droit du Travail), 12 (Documentation), 13 (Communication), 14 (Reporting), 15 (Budget), 16 (Diversite), 17 (Mobilité), 18 (RSE), 19 (Innovation), 20 (Pilotage), 22 (Avantages Sociaux) et 24 (Stagiaires) ont egalement des liens plus ou moins directs avec les donnees du Departement 1.

6. Plan de migration et etapes detaillees

6.1 Phase 0 : Preparation de l'environnement

Avant toute creation de table, l'environnement Supabase doit etre prepare. Cette phase critique comprend 5 etapes sequentielles qui doivent etre executees dans l'ordre. Premierement, la creation du schema parent admina et du schema enfant admina_rh via la commande `CREATE SCHEMA IF NOT EXISTS`. Deuxiemement, la configuration des permissions d'accès aux schemas pour le role `anon` (lectures uniquement via API) et `authenticated` (lectures/écritures via API). Troisiemement, l'installation de l'extension `pgcrypto` pour la generation d'UUID et l'extension `pg_stat_statements` pour le monitoring des requetes. Quatriemement, la creation de la table des tenants dans le schema admina. Cinquiemement, la configuration du parametre de session `app.tenant_id` via un database hook Supabase.

```
-- Phase 0 : Preparation
CREATE SCHEMA IF NOT EXISTS admina;
CREATE SCHEMA IF NOT EXISTS admina_rh;

GRANT USAGE ON SCHEMA admina TO anon, authenticated;
GRANT USAGE ON SCHEMA admina_rh TO anon, authenticated;

CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Table des tenants
CREATE TABLE admina.tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(200) NOT NULL,
  slug VARCHAR(100) UNIQUE NOT NULL,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

6.2 Phase 1 : Types ENUM et tables de reference

La premiere phase de migration consiste a creer l'ensemble des types ENUM et tables de reference necessaires au Departement 1. Sur les 39 listes de la feuille `_Lists`, environ 15 sont specifiques au Departement 1 et seront creees avec le prefixe `d1_`. Les 24 restantes sont partagees avec d'autres domaines et seront creees sans prefixe. La creation des ENUMs doit precéder la creation des tables car ces types sont references dans les definitions de colonnes. Chaque ENUM est cree avec la commande `CREATE TYPE` et peut etre altere ulterieurement avec `ALTER TYPE ... ADD VALUE` si de nouvelles valeurs sont necessaires.

ENUM specifique D1	Valeurs
d1_statut_demande	En attente, Validee, En cours, Pourvue, Annulee
d1_priorite	Urgente, Haute, Moyenne, Basse
d1_type_poste	Cadre, Agent de maitrise, Operationnel, Stagiaire
d1_motif_recrutement	Remplacement, Creation de poste, Saisonnalite, Surcharge
d1_source	Site web, Presse, Cooptation, Reseaux sociaux, Cabinet

ENUM specifique D1	Valeurs
d1_statut_candidat	Nouveau, En cours d'etude, Entretien planifie, Retenu, Refuse
d1_resultat_entretien	Favorable, Defavorable, A revoir, En attente
d1_decision_finale	Embauche recommandee, Embauche avec reserve, Refus, En attente
d1_stade_pipeline	CV reçu, Pre-selection, Entretien HR, Test technique, Offre, Embauche

Tableau 7 : Extrait des types ENUM specifiques au Departement 1

6.3 Phase 2 : Creation des tables principales (17 tables)

La seconde phase de migration cree les 17 tables SQL du Departement 1 dans l'ordre de dependance. Les tables sans cle etrangere sont creees en premier (tables de reference, cabinets, sources), puis les tables principales dans l'ordre du processus metier (demandes, candidats, entretiens, evaluations, verifications, contrats), et enfin les tables de suivi (integration, checklist, essai, formation, pipeline, post-embauche). Chaque table est creee avec ses contraintes NOT NULL, UNIQUE, CHECK et FOREIGN KEY. Les indexes sont ajoutes apres la creation des tables pour optimiser les requetes les plus frequentes (recherche par statut, par date, par departement).

6.4 Phase 3 : Index, triggers et fonctions

La troisieme phase ajoute la couche logique de la base de donnees. Les indexes sont cres sur les colonnes les plus frequemment utilisees dans les filtres et les jointures. Les triggers automatise la mise a jour des colonnes systeme `updated_at` et `version` a chaque modification. Une fonction de recherche full-text est creee sur les colonnes texte (nom, prenom, poste, competences) pour permettre la recherche globale de candidats. Un trigger de notification est mis en place pour emettre des evenements Realtime lorsque le statut d'une demande change, permettant au frontend de mettre a jour l'interface en temps reel.

```
-- Index recommandes
CREATE INDEX idx_d1_demandes_tenant_statut
ON admina_rh.d1_demandes_recrutement(tenant_id, statut);
CREATE INDEX idx_d1_candidats_tenant_nom
ON admina_rh.d1_base_candidats(tenant_id, nom, prenom);
CREATE INDEX idx_d1_entretiens_tenant_date
ON admina_rh.d1_planning_entretiens(tenant_id, date_entretien);
CREATE INDEX idx_d1_contrats_tenant_statut
ON admina_rh.d1_suivi_contrats(tenant_id, statut_contrat);

-- Trigger updated_at automatique
CREATE OR REPLACE FUNCTION admina_rh.trigger_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_demandes_updated
BEFORE UPDATE ON admina_rh.d1_demandes_recrutement
FOR EACH ROW EXECUTE FUNCTION admina_rh.trigger_updated_at();
```

6.5 Phase 4 : Politiques RLS (17 tables)

La quatrieme phase active le Row Level Security sur chacune des 17 tables et cree les politiques d'accès pour les 5 roles. Chaque table recoit au minimum 4 politiques : SELECT, INSERT, UPDATE, DELETE. Les politiques sont generees automatiquement par un script Python qui parcourt la matrice des permissions (Tableau 5) et genere le SQL correspondant. Le nombre total de politiques RLS pour le Departement 1 est d'environ 68 (17 tables x 4 operations), mais certaines tables comme `d1_pipeline_candidatures` peuvent avoir des politiques supplementaires pour gerer les transitions de stade. Toutes les politiques incluent le filtre `tenant_id = current_setting('app.tenant_id')::uuid` comme premiere condition, garantissant l'isolation multi-tenant.

6.6 Phase 5 : Donnees de test et validation

La cinquieme et derniere phase de migration consiste a injecter des donnees de test representatives et a valider le bon fonctionnement de l'ensemble. Les donnees de test sont extraites directement des fichiers Excel existants (les lignes deja presentes dans les 18 feuilles). Chaque jeu de donnees est injecte avec un `tenant_id` specifique de test, puis des requetes de validation verifient : (1) l'isolation multi-tenant (un utilisateur du tenant A ne peut pas voir les donnees du tenant B), (2) les contraintes d'integrite referentielle (impossible d'insérer un candidat avec un statut invalide), (3) les triggers de mise a jour automatique (`updated_at` se met a jour correctement), (4) les performances des requetes (les indexes sont utilises, temps de reponse inferieur a 100ms pour les requetes standard), et (5) la coherence des donnees migrees par rapport aux fichiers Excel sources.

7. Estimation de la charge et roadmap

7.1 Volumetrie du Departement 1

Le Departement 1 represente un volume significatif de travail de migration. Avec 17 tables SQL, 39 types ENUM, 8 indexes, 68 politiques RLS, et environ 250 colonnes au total, la creation complete du schema est un projet en soi. L'estimation ci-dessous prend en compte l'écriture du SQL, les tests unitaires, la validation des donnees migrees et la documentation. Le total estime est d'environ 5 a 7 jours de travail pour un developpeur familierise avec PostgreSQL et Supabase.

Composant	Quantite	Unite	Temps estime
Types ENUM (specifiques D1)	15	types	2 heures
Types ENUM (partages)	24	types	3 heures
Tables SQL	17	tables	1 jour
Colonnes systeme (auto)	17 x 8	colonnes	1 heure (script)
Index	8 a 12	indexes	2 heures
Triggers (<code>updated_at</code>)	17	triggers	1 heure (script)
Fonctions (<code>search</code> , <code>notif</code>)	3 a 5	fonctions	3 heures
Politiques RLS	~68	politiques	1 jour
Donnees de test	~500	lignes	4 heures
Validation & QA	1	lot	1 jour

Composant	Quantite	Unite	Temps estime
Documentation technique	1	document	4 heures

Tableau 8 : Estimation detaillee de la charge de migration

7.2 Roadmap de mise en oeuvre

La mise en oeuvre du Departement 1 dans Supabase se fera en 5 phases sequentielles, avec des jalons de validation entre chaque phase. La Phase 0 (Preparation) prend une demi-journee et produit les schemas et la table des tenants. La Phase 1 (ENUMs) prend une demi-journee et produit les 39 types ENUM. La Phase 2 (Tables) prend 2 jours et produit les 17 tables avec leurs contraintes. La Phase 3 (Logique) prend 1 jour et produit les indexes, triggers et fonctions. La Phase 4 (RLS) prend 1 jour et produit les 68 politiques de securite. La Phase 5 (Validation) prend 1 jour et valide l'ensemble avec des donnees de test. Le total est de 5 a 7 jours ouvrables, avec une marge de securite pour les ajustements et les corrections eventuelles. Chaque phase produit un fichier de migration SQL versionne (001, 002, 003, etc.) qui peut etre annule avec `supabase db reset` en cas de probleme.

7.3 Risques et mitigations

Plusieurs risques ont ete identifies pour la migration du Departement 1. Le risque principal est la perte de donnees lors de la conversion Excel vers SQL, mitige par les 4 couches de detection d'erreurs documentees dans le PDF de methodologie (validation de type, validation de domaine, validation de reference, validation croisee). Un deuxieme risque concerne la performance des requetes sur les tables volumineuses (`base_candidats` avec 37 colonnes), mitige par la creation d'indexes strategiques et l'utilisation de vues partielles pour les requetes frequentes. Un troisieme risque porte sur la complexite des politiques RLS qui peuvent degrader les performances si elles sont mal ecrites, mitige par l'utilisation de fonctions d'aide (helper functions) et de tests de performance systematiques. Enfin, le risque de regression lors de l'evolution des ENUMs est mitige par l'utilisation de types ENUM modifiables et par des scripts de migration incrementale.